

Ledger Store

Feature & Component Architecture Report

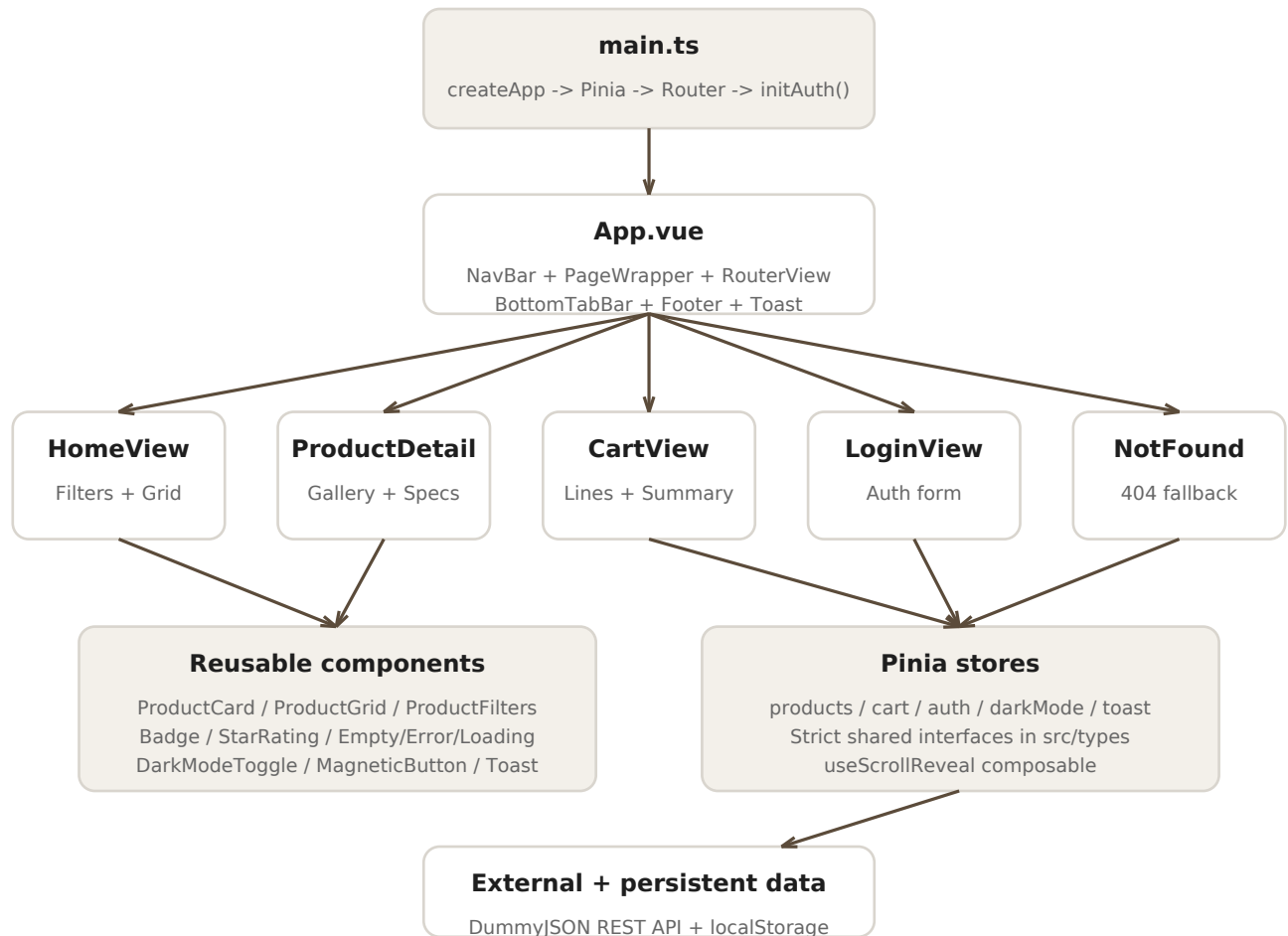
Project type	Data-driven Vue 3 Single Page Application (SPA)
Stack	Vue 3 Composition API, TypeScript, Vite, Tailwind CSS, Pinia, Vue Router
Data source	DummyJSON products and authentication endpoints
Domain	Curated e-commerce catalog for menswear (shirts, shoes, watches)

1. Features Implemented

- Live product catalog - Fetches the complete DummyJSON product collection and category list, then curates the catalog to mens-shirts, mens-shoes, and mens-watches.
- Search - Case-insensitive search across product title, description, and brand.
- Filtering and sorting - Category filtering, minimum/maximum price filtering, and sorting by ascending/descending price, rating, or product name.
- Dynamic routing - Routes for home, category, product detail, cart, login, and a catch-all 404 page. Product/detail and other secondary views are lazy-loaded.
- Product detail view - Dedicated detail route with image gallery, price/discount information, stock/availability badges, specifications, shipping/return/warranty information, ratings, and reviews.
- Authentication - Uses DummyJSON /auth/login and /auth/me. The access token is stored in localStorage, restored on startup, and removed on logout or failed session validation.
- Protected cart route - A Vue Router navigation guard redirects unauthenticated users to login and returns them to the requested route after successful login.
- Shopping cart - Add, remove, update quantities, clear cart, item count, subtotal/discount/shipping/total calculations, and per-user cart persistence in localStorage.
- Toast feedback - Global success/info/error notifications with automatic dismissal, used for cart and authentication interactions.
- Dark mode - System-preference aware theme, manual toggle, Tailwind dark variants, and persistent preference in localStorage.
- Responsive UI - Responsive product grid, desktop navigation, mobile menu, and a dedicated mobile bottom tab bar.
- Loading, error and empty states - Skeleton grid/detail placeholders, spinner, reusable error display, retry flow, and empty-result state.
- Interaction polish - Page transitions, scroll-reveal behavior, product-card motion, custom desktop cursor, magnetic button effect, and hover/focus feedback.
- Strict TypeScript model - Shared Product, AuthUser, CartLine, FilterState, PaginatedResponse and union types are used instead of any in application code.

2. Component Hierarchy

The application uses route-level views for pages, small reusable components for presentation/interaction, and focused Pinia stores for shared state. App.vue acts as the visual shell, while Vue Router selects the active view.



Hierarchy notes

- HomeView - ProductFilters -> ProductGrid -> ProductCard; ErrorState handles fetch failure.
- ProductDetailView - Badge + StarRating + detail/loading/error UI; reads :id from Vue Router.
- CartView - LoadingSpinner + EmptyState + MagneticButton; combines cart lines with product data.
- Global shell - NavBar (with DarkModeToggle), BottomTabBar, Footer, Toast, and PageWrapper surround RouterView.
- Shared logic - Pinia stores provide products, auth, cart, dark mode, and toast state; useScrollReveal is a reusable composable.

3. Architectural Explanation

Presentation layer. Route-level views are intentionally separated from reusable UI components. Views coordinate routing and store state; components focus on rendering and localized interaction. This prevents App.vue from becoming a single large component and makes product cards, filters, feedback states, rating display, and navigation reusable.

State layer. Shared application state is split across five Pinia stores: products, cart, auth, darkMode, and toast. Each store has one responsibility and exposes computed state plus explicit actions. The products store centralizes fetching, caching, filtering, and sorting; the cart store manages user-specific cart lines; the auth store owns session restoration and login/logout; the dark-mode store persists theme preference; and the toast store manages transient feedback.

Routing layer. Vue Router maps URLs to route-level views. Dynamic parameters are used for categories and product IDs. A beforeEach guard protects /cart, while scrollBehavior preserves the user's position when switching between home/category routes. Lazy loading is used for secondary views to keep the initial route smaller.

Data and persistence. Remote product and authentication data comes from DummyJSON through fetch. Authentication tokens, per-user cart data, and the dark-mode preference are stored in localStorage. On startup, main.ts waits for auth.initAuth() before mounting so persisted sessions can be validated first.

Type safety. API and application data structures are declared in src/types/index.ts. Generic PaginatedResponse<T>, Product, AuthUser, CartLine, FilterState, SortOption, and toast types are referenced across stores and components, keeping the data flow explicitly typed.

Data Flow

Step	From	To	Purpose
1	DummyJSON	products store / auth store	Typed API responses are fetched.
2	Pinia stores	Views + components	Reactive state is consumed through computed values and actions.
3	User interaction	Store actions / Router	Search, filters, cart actions, login, navigation, and theme changes update state.
4	localStorage	auth / cart / darkMode	Selected state persists across reloads and is restored when the app starts.

Summary

The hierarchy follows a clear shell -> route view -> reusable component pattern, with shared state isolated in Pinia stores and API/persistence concerns kept outside most presentation components. This structure directly supports the assignment requirements for reusable components, strict TypeScript, asynchronous REST data, responsive Tailwind UI, dynamic detail navigation, and bonus features such as authentication, persistent cart state, routing, and dark mode.